Source: docs\WISHLIST_BAG_DESIGN.md | Export: 11 November 2026

Wishlist, Shopping Bag and customer notifications

Local implementation of the four supplied Stitch frames. The existing storefront shell, approved logo, local Lora font, customer authentication and trusted shopping/checkout services are retained. No production data, uploads, rules, indexes or deployments were changed.

Implemented screens

- `src/views/WishlistView.tsx`: compact two-column mobile/four-column desktop cards; constrained 3:4 media; saved-heart removal; product name with actual category below it; current price and valid optional MRP; Add to Bag, required Select Colour, View Bag and unavailable states. Small collections retain normal card widths. Available entries appear before unavailable entries, without changing saved data.
- `src/views/BagView.tsx`: desktop item/summary columns and compact mobile rows; selected colour, unit price, desktop line amount, quantity controls and confirmed removal. Unavailable entries remain visible/removable and excluded from the purchasable subtotal. The mobile checkout dock sits above the existing bottom navigation, with content/safe-area padding so the last item and footer remain reachable.
- `src/shopping/ShoppingDialogs.tsx`: native desktop dialogs/mobile sheets for required colour selection and removal. No colour is preselected. Exact product/variant membership determines View Bag, including a previously added colour whose stock is now zero. Unavailable colours cannot be newly added. Removal requires confirmation, keeps item context, exposes progress/retry and restores focus (or the page heading when the trigger is removed). Shop/Product Details Wishlist removals reuse the same confirmation through the shared state.
- `src/shopping/CustomerNotice.tsx`, `feedback.ts`, `shopping.css`: one compact customer notification below the header, with a restrained icon, close button and at most one appropriate destination or Retry. Success dismisses after four seconds; hover and keyboard focus pause the remaining time independently. Replacement/unmount clears the old deadline. Errors stay until resolved/dismissed. Mobile reserves the notification's measured height so essential page controls are not covered. Reduced motion is respected; notifications do not steal focus.

Authoritative values and identity

The existing `shopping:get`, `bag:add`, `bag:adjust`, `bag:remove` and `wishlist:set` operations remain authoritative. Server ownership comes from the verified token UID; the browser never supplies an owner UID. The existing Firestore rules permit an owner to read their own Bag/Wishlist references and deny direct client writes. Administrator authorization still requires both its claim and enabled access record.

The API resolves current published products and enabled categories in batches. Covers, names, category names, prices, MRP, variants and quantities use that returned public catalog data. Hidden/deleted products retain removable references with neutral unavailable/image fallbacks; private metadata is not exposed to fill visual gaps. Images use explicit bounds and preserve proportions.

Wishlist remains a unique-product collection. Bag identity is product plus exact nullable variant, using the existing `bagId` convention. Adding to Bag **does not remove the Wishlist entry**, preserving the existing behavior. Badge definitions remain Wishlist unique-product count and Bag summed quantity, including unavailable saved entries. The summary separately labels purchasable units and subtotal.

Mutation and request safety

`src/shopping/useShopping.ts` retains its existing authenticated state/persistence and receipt-based mutations. Changes include:

- Failed refreshes retain previously loaded canonical items and badges, with explicit failure/Retry rather than a false empty collection.
- Per-product mutation locks prevent duplicate/conflicting taps while unrelated products remain usable.
- Uncertain acknowledgements retain the **same operation ID and body** for safe replay. Affected entries stay blocked until the issue is resolved. Receipt replay cannot blindly apply an additive quantity change again.
- Confirmed writes followed by a failed refresh use read-only reconciliation; the already-confirmed mutation is not resubmitted. Definite stock/availability rejections similarly require a refresh/review before another change.
- Persistent per-item recovery survives replacement/dismissal of the global toast. Dismissing feedback does not discard an unresolved operation.
- UID/generation guards reject stale results; requests abort on unmount/account change. Navigation clears stale notifications and removal prompts; late completion from an earlier navigation does not resurrect its toast. Bag/Wishlist local dialogs are keyed to the current UID.
- Success feedback follows confirmed API success and reconciliation. Wishlist actions never offer View Bag; current-page destinations are omitted. Pending feedback appears beside the affected control, including the existing Shop/Product Details integration.

Checkout boundary

The summary says **Items Subtotal**, not final payable total. Shipping is calculated at checkout; coupons and eligible loyalty points are applied there. There are no invented shipping/tax/savings values or stock reservations.

Bag checkout continues to `/checkout` without a Buy Now draft parameter. The dock/summary block when no line is purchasable, a Bag mutation is active/unresolved, or canonical loading failed. The small `useCheckoutReview.ts` and `CheckoutView.tsx` changes also block a direct Bag-checkout route from using a stale review while a Bag update is pending. Buy Now ignores unrelated Bag pending state and keeps its independent source. Existing trusted quotes, paise totals, pre-coupon/strictly-above-threshold shipping, coupons, loyalty and order validation remain unchanged.

Necessary differences from Stitch

The existing header/footer/navigation and real logo remain. Category text is below product names, as requested, instead of overlaying photographs. Unsupported certificates, insurance/video appointments, urgency, payment logos, tax/packaging claims and prototype simulators are omitted. Unavailable private products use neutral fallbacks. The Bag count follows the existing quantity contract; unavailable rows are excluded only from the purchasable subtotal. No Move to Wishlist, bulk actions or unsafe Undo were added.

Verification

- `npm run lint`, `npm test` (148 tests), `npm run build`; build retains its pre-existing large-chunk advisory.
- Existing exported shopping API emulator suite: real unverified-user authentication, owner isolation, forbidden direct writes, duplicate prevention, operation replay, variants, quantity/stock changes, category visibility and safe removal.
- Existing exported order emulator suite: customer history ownership plus COD receipts, stock/collection/cancellation security and Buy Now/Bag isolation. Three top-level API tests passed across the two suites.
- `scripts/shopping-design-browser.test.py`: real loopback APIs, mandatory colour selection, exact variant membership, unavailable removal, success/pause/dismissal and action-specific links, canonical badges, lost acknowledgement with same receipt, rapid taps, independent-item controls, stock reduction/recovery, initial versus later load failures, per-item retry, shared Product Details confirmation, empty/unavailable states, Bag checkout handoff and stale-account cleanup.
- Existing checkout-design browser suite: all three stages at four widths, address dialogs, authoritative totals, coupon/points behavior, price acknowledgement, stock blocking and no order creation on reads.
- New unit coverage tests the notification mapping and deterministic timeout/hover/focus/replacement/disposal lifecycle. Existing unit tests cover exact subtotals, quantity bounds, unavailable exclusions, shipping and checkout sources.

Visual checks cover **320, 390, 768, 1440 and 1920 CSS pixels**. `.cache/shopping-design/` contains `wishlist-{width}.png`, `bag-{width}.png`, colour/removal sheets, `toast-{390,1440}.png` and equivalent-width `comparison-{wishlist,bag}-{390,1440}.png` reference comparisons. Screenshots use isolated emulator products and local fixture photography served only by the test browser. Actual application media always comes from its existing product records.

Release

Frontend release only: there are no backend contract, dependency, environment, rules, index or data-migration changes. Hosted acceptance remains after the owner's normal release, using existing customer accounts and products. This local verification does not establish production deployment or live connectivity.

Nothing was committed, pushed, deployed, published, uploaded to production or changed in production.